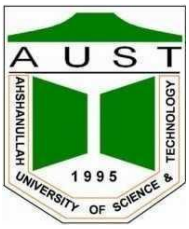
Ahsanullah University of Science & Technology
Department of Computer Science & Engineering



TV Nights on the Couch

Computer Graphics Lab (CSE 4204)

Final Project Report

Submitted By:	
Tahsin Shuborna	20210204064
Shahriar Hossain Arafat	20210204058

1. Problem Analysis

1.1 Project Requirements

As part of the *CSE4204: Computer Graphics Lab* final project, our objective is to design and implement an interactive 3D scene using Three.js. Our group was assigned Project 5, which is to create a 3D scene of a couch and a lamp.

The project must satisfy the following core requirements:

- Implementation of custom shaders to control object appearance
- Use of lighting models to simulate realistic illumination
- Application of perspective projection for depth perception
- Adding textures to all objects in the scene
- Inclusion of animation to enhance visual dynamics
- Support for mouse and keyboard interaction for user control

In addition to these general requirements, the assigned project includes the following specific features:

- A couch with appropriate texture mapping
- A lamp stand with texture
- The camera will move around the couch, enabling scene exploration
- The texture of the couch will dynamically change based on interaction
- The lamp light will gradually turn on and off, creating a smooth animation effect

We tried our best to make the project as good as possible by properly implementing all the required features.

1.2 Core Integration

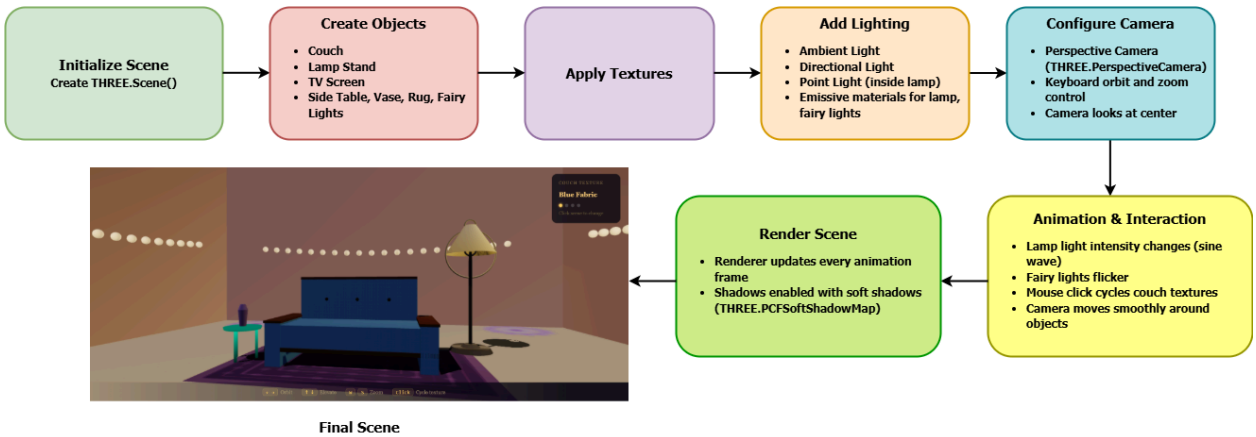


Fig: Virtual Couch & Lamp Construction Steps

In this project, the models are designed using basic geometries in Three.js instead of importing external models. The couch has been built by combining multiple box geometries for the seat, backrest, armrests and legs using a group (THREE.Group). The lamp stand is created using cylinder, sphere and torus geometries to represent the base, pole, shade and bulb.

For the camera, a perspective camera (THREE.PerspectiveCamera) has been used to create a realistic 3D view. The camera moves around the couch using keyboard controls, allowing the user to explore the scene from different angles.

Lighting is implemented using different types of lights such as ambient light, directional light and point lights. A point light is placed inside the lamp to simulate real lighting. Its intensity changes smoothly over time, so the lamp slowly turns on and off. In addition, fairy lights are added around the scene, which also glow and change intensity together with the lamp, making the environment more visually attractive.

For texturing, procedural textures are created using canvas (CanvasTexture) and applied to the couch, lamp, floor and walls. The couch has multiple textures, and users can change the texture by clicking, which adds interactivity.

To enhance the project further, we have added extra objects like a TV screen in front of the couch to create a more realistic living room environment, where a user can imagine sitting and watching something. Decorative elements such as a side table, vase, rug and fairy lights are also included to improve the overall aesthetics of the scene.

All models, camera, lighting, textures and extra features are combined together to build a complete and interactive 3D scene, basically turning our code into a small virtual living room.

1.3 Technical Challenges

One of the main challenges in this project was keeping all objects properly placed while allowing the camera to move freely. Sometimes the camera would sneak too close to the couch or lamp, or the lighting would produce weird shadows, making the scene look unnatural. Careful mathematical planning of camera, object transformations, lighting and textures was key to solving this.

Camera Movement

The camera uses spherical coordinates to orbit around the scene:

$$x = r \cdot \sin\theta \cdot \cos\phi$$

$$y = r \cdot \sin\phi$$

$$z = r \cdot \cos\theta \cdot \cos\phi$$

Where:

- θ = horizontal rotation
- ϕ = vertical tilt
- r = distance from the center of the scene

After calculating the camera position, we set it to look at the center with:

camera.lookAt(0, 1.6, 0);

This ensures the camera smoothly orbits without losing focus on the scene's main objects.

Object Transformations

Objects like the couch, lamp and TV are constructed from basic geometries (boxes, cylinders, spheres). Each piece is positioned manually in 3D:

mesh.position.set(x, y, z);

Grouping with THREE.Group() allows moving all parts of an object together. Mathematically, this applies hierarchical transformations, where the world position of a child is:

$$M_{world} = M_{parent} \cdot M_{local}$$

This made it easier to maintain relative positions when moving complex objects like the lamp or couch.

Lighting

- Lamps and bulbs use emissive materials, which mathematically add light color to surfaces, independent of scene lighting:

$$Color_{final} = Color_{diffuse} + Color_{emissive}$$

- Shadows are enabled and softened:

renderer.shadowMap.enabled = true;

renderer.shadowMap.type = THREE.PCFSoftShadowMap;

Careful positioning of lights ensured natural looking illumination without clipping shadows.

Textures

Procedural textures (ouch fabric, floor tiles) are drawn on a canvas and mapped using UV coordinates. The repeat property adjusts how patterns tile over surfaces:

texture.repeat.set(2, 2);

Aspect ratios are preserved to prevent distortion, especially on flat surfaces like the TV screen or floor.

Challenges Overcome

It was tricky to:

- Keep the camera at a comfortable distance without clipping into objects
- Maintain realistic lighting and soft shadows
- Ensure textures looked correct across all objects

By carefully calculating camera angles, object positions, texture mapping and lighting, the scene became smooth, interactive and visually appealing.

2. User Interaction

2.1 Key Interaction

Keyboard interaction is implemented using event listeners that track key presses and update camera parameters dynamically.

In the code, a keys object stores the current state of pressed keys:

```
window.addEventListener('keydown', e => { keys[e.key] = true; });

window.addEventListener('keyup', e => { keys[e.key] = false; });
```

These inputs are processed inside the animation loop to continuously update the camera.

The camera movement is controlled using spherical coordinates:

Left / Right Arrow (← →)
These keys modify the horizontal angle camTheta:

```
camTheta -= 0.024; // left

camTheta += 0.024; // right
```

This rotates the camera around the scene, creating an orbiting effect.

Up / Down Arrow (↑ ↓)
These keys adjust the vertical angle camPhi:

```
camPhi += 0.016; // up

camPhi -= 0.016; // down
```

This allows the user to look up and down, changing the elevation.

W / S Keys
These control the camera distance (camRadius):

```
camRadius -= 0.12; // zoom in
```

camRadius += 0.12; // zoom out

This creates a zoom in and zoom out effect.

After updating these values, the camera position is recalculated using spherical-to-Cartesian transformation:

positionCamera();

Additional Features

The camera is always focused on the center of the scene which ensures the main objects remain visible during movement.

camera.lookAt(0, 1.6, 0);

Constraints are applied to prevent unnatural motion:

- camPhi is limited to avoid flipping
- camRadius is clamped to prevent clipping into objects

Usability
This control scheme allows smooth and intuitive navigation. The user can fully explore the 3D scene by rotating, tilting and zooming the camera without losing focus on the central objects (couch and lamp).

2.2 Mouse Interaction

Mouse interaction is implemented using a click event on the rendering canvas:

renderer.domElement.addEventListener('click', () => {

texIdx = (texIdx + 1) % TEXTURES.length;

});

Each click cycles through different couch textures stored in the TEXTURES array. When the texture changes, a new material is created. It is applied to all couch fabric meshes. The UI (texture name and indicator dots) is updated and a brief flash effect is triggered for visual feedback

Usability:
This interaction provides a simple and engaging way for users to customize the scene. With just a click, users can instantly preview different couch materials (fabric, velvet, leather), enhancing interactivity without complex controls.

Additional Interaction

Apart from user input, the scene also includes time based interaction through animation:

const cycle = (Math.sin(clock * 1.05) + 1) / 2;

lampLight.intensity = cycle * MAX_INTENSITY;

- The lamp light smoothly turns on and off over time
- The bulb and lampshade emissive intensity also change dynamically
- Fairy lights flicker based on the same cycle

Usability:
This adds realism and atmosphere to the scene, making it feel more dynamic even without user input.

3. Design and Aesthetic Aspects

3.1 Visual Elements

Fig: Lamp, TV, Bottle on side Table, Floral Floor

In our project, we focused on creating a visually appealing and immersive indoor environment by combining procedural textures, multiple lighting techniques and carefully designed objects.

Texture Design

We have implemented procedural textures using HTML canvas instead of relying on external image files. This allowed us to have full control over patterns and appearance.

- **Couch Textures**
  We designed multiple fabric styles such as **blue woven fabric, red velvet, beige linen and green leather**.
  These textures are created using grid patterns, gradients and noise to simulate realistic

materials.
Additionally, users can dynamically switch between these textures using mouse interaction.

- **Floor Texture**
  We created a **beige base floor with a lavender floral pattern** to give a decorative and aesthetic look.
  Random noise has been added to avoid a flat appearance.
- **Walls**
  The walls are given a **soft plaster like texture** with subtle noise variation to maintain a calm indoor ambiance.
- **Lamp Materials**
  The lamp stand uses a **metallic texture with gradient shading**, while the lampshade uses a **fabric texture with vertical patterns**, making it look soft and realistic.
- **Rug Design**
  A patterned rug was placed under the couch using layered colors and geometric shapes to enhance the interior decoration.

Additional Visual Objects

We also included several elements to enrich the scene:

- **TV Screen (Media Display)**
  A TV is placed in front of the couch displaying an image (Argentina World Cup Win).
  Texture mapping is adjusted using aspect ratio correction to prevent distortion, ensuring the image fits properly on the screen.
- **Fairy Lights**
  Decorative fairy lights are placed along the walls using small emissive spheres combined with point lights. These lights create a warm and cozy atmosphere.
- **Vase (Decorative Object)**
  A small vase is placed on a side table to add realism and detail to the environment.
- **Side Table**
  A custom designed table using shader material adds variation and uniqueness to the scene.

Lighting Model

We have used multiple lighting models to achieve realistic shading:

- **Lambertian Reflection Model**
  Most objects use THREE.MeshLambertMaterial, which provides diffuse lighting based on the angle between light and surface.
- **Standard Material (PBR)**
  The bulb uses THREE.MeshStandardMaterial with properties like roughness and metalness for more realistic rendering.
- **Emissive Materials**
  The lamp bulb, fairy lights and parts of the lampshade use emissive properties, allowing them to glow independently.

Lighting Setup

To enhance realism, we have combined multiple light sources:

- **Ambient Light:** Provides base illumination
- **Directional Light:** Simulates soft environmental lighting
- **Point Light (Lamp):** Main dynamic light source
- **Fill and Rim Lights**: Improve depth and visual contrast

We have also enabled soft shadows using:

renderer.shadowMap.enabled = true;

renderer.shadowMap.type = THREE.PCFSoftShadowMap;

Dynamic Aesthetic Effects

We have implemented time based animation for lighting:

- The lamp light smoothly turns on and off using a sine function
- Fairy lights flicker dynamically
- Emissive intensity of objects changes over time

This creates a living, dynamic environment rather than a static scene.

Overall Aesthetic Outcome

By combining procedural textures, dynamic lighting, decorative objects and interactive elements, we have successfully created a cozy and realistic 3D living room environment. The inclusion of elements like the TV, fairy lights, rug and decorative vase enhances both realism and user engagement.

3.2 Snapshots

The following snapshots show the side, top, front views of the 3D living room scene.

1. Couch Model

- The couch is created using THREE.Group, combining multiple geometries such as:
    - seat cushion
    - backrest
    - armrests
    - legs

2. Lamp and Shadow Effect

- A floor lamp is placed beside the couch.
- The lamp consists of:
    - cylinder (stand)
    - cone/cylinder (shade)

    - emissive bulb

A PointLight is used inside the lamp, producing:

- soft illumination
- realistic shadow on the floor

The shadow visible on the floor confirms shadow mapping is enabled.

3. Fairy Lights (Decorative Lighting)

- Multiple small bulbs are arranged along the wall.
- Each bulb uses:
    - SphereGeometry (visual bulb)
    - PointLight (light source)

Lights are positioned using a sine based sag function, creating a natural hanging effect.

4. TV Screen (Texture Mapping)

- A PlaneGeometry is used as the TV screen.
- An image texture is mapped onto it using TextureLoader.

Aspect ratio is preserved by adjusting:

- texture.repeat
- texture.offset

This ensures the image is not stretched.

5. Camera & Perspective

- The scene is viewed from a side angle, controlled by:
    - Arrow keys → rotation & elevation
    - W/S → zoom

The camera uses Perspective Projection:

- Objects farther away appear smaller
- Creates realistic depth

6. Environment & Depth

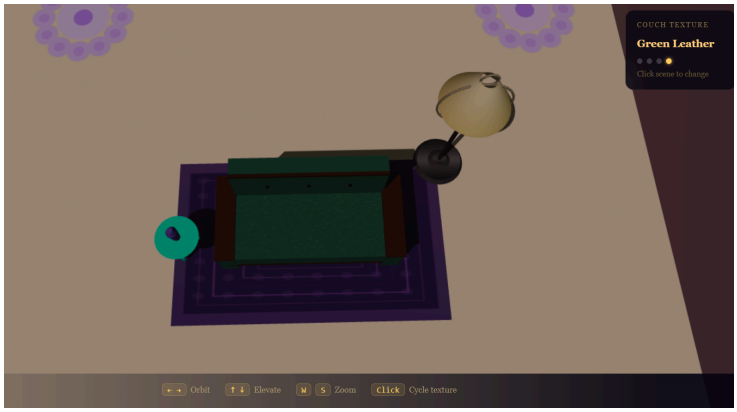
- Walls and floor create an enclosed room
- Fog is applied to enhance depth perception

7. Lighting System

Multiple light sources are used:

- Ambient light → overall brightness
- Directional light → soft global illumination
- Point light (lamp) → main focus light

This combination creates a realistic lighting environment.



4. Code Implementation

4.1 Software Platform

In our project, we have used a combination of web based technologies and graphics libraries to implement the interactive 3D scene.

- **HTML5**
  We have used HTML to structure the web page and integrate the rendering canvas where the 3D scene is displayed.
- **CSS3**
  CSS is used to design the user interface elements such as the HUD (heads up display), control panel and overall visual styling of the page.
- **JavaScript (ES6)**
  JavaScript was used as the core programming language to implement all logic, including scene creation, object transformations, animations and user interactions.
- **Three.js**
  We used the Three.js library to simplify WebGL based 3D rendering. It provides built-in classes for creating scenes, cameras, geometries, materials, lighting and animations.
- **WebGL**
  WebGL is the underlying graphics API used by Three.js to render 3D graphics directly in the browser using the GPU.
- **GLSL (OpenGL Shading Language)**
  GLSL is used for implementing custom shader effects, such as the gradient based shader applied to the side table.

- **Canvas API**
  Used to generate procedural textures dynamically (e.g., couch fabric, floor patterns).
- **Event Handling System**
  JavaScript event listeners are used to handle keyboard and mouse interactions.
- **Browser Environment**
  The project has been developed and tested in VS Code using the Live Server extension, allowing instant preview of changes without any additional software installation. Opening the HTML file via the live server launches the interactive 3D scene immediately.

4.2 Custom Shaders

In this project, custom shader logic is implemented via **Three.js materials**, simulating vertex and fragment shader behavior:

- **Vertex Shader Logic:**
  Transforms each object's vertices from local space → world space → camera space, passing normals and texture coordinates for lighting and texturing.
- **Fragment Shader Logic:**
  Determines pixel color based on lighting, emissive glow and texture mapping. Key effects include:
  - Diffuse & ambient lighting for realistic shading.
  - Emissive colors for glowing objects (lamp bulb, couch highlights).
  - Procedural textures generated on canvas, applied via map (e.g., couch fabrics, floor, rug, lampshade).

Object	Material / Shader Logic
Couch	MeshLambertMaterial with procedural fabric texture + emissive color
Lamp Shade	MeshLambertMaterial with procedural lampshade texture + slight emissive

Lamp Bulb	MeshStandardMaterial with high emissive for glow effect
Floor & Rug	MeshLambertMaterial with procedural textures
TV Screen	MeshBasicMaterial with loaded image texture

Implementation Example:

```
const fab = new THREE.MeshLambertMaterial({ map: T_BLUE, emissive: new THREE.Color(0x040e20) });

const bulbMat = new THREE.MeshStandardMaterial({
  emissive: new THREE.Color(0xffcc44),
  emissiveIntensity: 3.5,
  color: new THREE.Color(0xffe880),
  roughness: 0.2, metalness: 0.0
});
```

This approach achieves realistic lighting, surface detail, and glow effects without manually writing GLSL code.

4.3 Project Feature Table:

The table below summarizes the features implemented in our Couch & Lamp project, classified into three categories based on their completion status.

#	Features	Status
1	Couch with procedural fabric textures	Implemented
2	Lamp with glowing bulb and lampshade	Implemented
3	Floor with procedural wood + floral rug	Implemented
4	TV with image texture and black frame	Implemented
5	Interactive camera orbit, zoom, elevation	Implemented
6	Texture cycling on mouse click	Implemented

7	Wall and environment procedural textures	Implemented
8	Advanced shader effects (custom GLSL)	Partially Implemented
9	Real time shadows on all objects	Partially Implemented
10	Physics or collision detection	Not Implemented

Table 01: Project Feature Table

5. Contribution

The project work has been divided between the team members as follows:

- **Shahriar Hossain Arafat (20210204058):** Structured the project, organized the scene hierarchy, grouped geometries, implemented the camera movement and ensured proper animation for lamp light and interactive features.
- **Tahsin Shuborna (20210204064):** Implemented the custom shaders controlling object appearance, integrated lighting models for realistic illumination, applied textures to all objects (couch, lamp, rug, floor, walls), set up the TV screen with image mapping, designed the overall scene background and aesthetic elements, managed color schemes and visual design details, prepared project report.

Both members collaborated to ensure the project meets all core requirements. We collaborated closely to properly implement all required features, making the project as complete and polished as possible.